

Scenario Based Problems

Experiment 1: NumPy Array Operations

Aim:

To find the average sale price of houses having more than 4 bedrooms using NumPy arrays.

Algorithm:

1. Import NumPy.
2. Store the dataset in a NumPy array.
3. Filter houses with bedrooms > 4.
4. Calculate the average sale price.
5. Display the result.

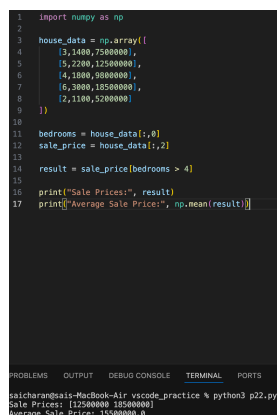
Code:

```
import numpy as np

house_data = np.array([
    [3,1400,7500000],
    [5,2200,12500000],
    [4,1800,9800000],
    [6,3000,18500000],
    [2,1100,5200000]
])

bedrooms = house_data[:,0]
sale_price = house_data[:,2]
result = sale_price[bedrooms > 4]
print("Sale Prices:", result)
print("Average Sale Price:", np.mean(result))
```

Output:



```
1 import numpy as np
2
3 house_data = np.array([
4     [3,1400,7500000],
5     [5,2200,12500000],
6     [4,1800,9800000],
7     [6,3000,18500000],
8     [2,1100,5200000]
9 ])
10
11 bedrooms = house_data[:,0]
12 sale_price = house_data[:,2]
13 result = sale_price[bedrooms > 4]
14
15 print("Sale Prices:", result)
16 print("Average Sale Price:", np.mean(result))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@saicharan-MacBook-Air: ~/vscode_practice % python3 p22.py

Sale Prices: [12500000 18500000]

Average Sale Price: 15500000.0

Result:

Thus, the average sale price of houses with more than four bedrooms is 15,500,000.

Experiment 2: Logistic Regression - Spam Detection

Aim:

To classify emails as Spam or Not Spam using Logistic Regression.

Algorithm:

1. Import libraries.
2. Prepare feature matrix and labels.
3. Train Logistic Regression model.
4. Accept user input.
5. Predict and display the result.

Code:

```
from sklearn.linear_model import LogisticRegression

X=[[1,3,120],[6,15,300],[0,1,80],[4,10,250],[2,5,160]]
y=[0,1,0,1,0]

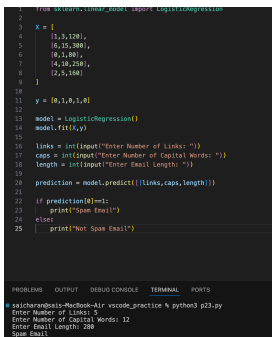
model=LogisticRegression()
model.fit(X,y)

links=int(input("Enter Number of Links: "))
caps=int(input("Enter Number of Capital Words: "))
length=int(input("Enter Email Length: "))

prediction=model.predict([[links,caps,length]])

if prediction[0]==1:
    print("Spam Email")
else:
    print("Not Spam Email")
```

Output:



```
1 from sklearn.linear_model import LogisticRegression
2
3 X = [
4     [1,3,120],
5     [6,15,300],
6     [0,1,80],
7     [4,10,250],
8     [2,5,160]
9 ]
10 y = [0,1,0,1,0]
11
12 model = LogisticRegression()
13 model.fit(X,y)
14
15 links = int(input("Enter Number of Links: "))
16 caps = int(input("Enter Number of Capital Words: "))
17 length = int(input("Enter Email Length: "))
18
19 prediction = model.predict([links,caps,length])
20
21 if prediction[0]==1:
22     print("Spam Email")
23 else:
24     print("Not Spam Email")
25
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

vscode@vscode:~/python3\$ python3 p2.py
Enter Number of Links: 1
Enter Number of Capital Words: 12
Enter Email Length: 208
Spam Email

Result:

Thus, the Logistic Regression model successfully classified the email.

Experiment 3: Logistic Regression (Duplicate)

Aim:

To classify emails as Spam or Not Spam using Logistic Regression.

Algorithm:

1. Import libraries.
2. Prepare feature matrix and labels.
3. Train Logistic Regression model.
4. Accept user input.
5. Predict and display the result.

Code:

```
from sklearn.linear_model import LogisticRegression

X=[[1,3,120],[6,15,300],[0,1,80],[4,10,250],[2,5,160]]
y=[0,1,0,1,0]

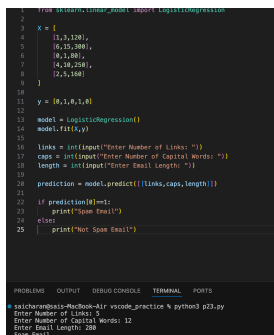
model=LogisticRegression()
model.fit(X,y)

links=int(input("Enter Number of Links: "))
caps=int(input("Enter Number of Capital Words: "))
length=int(input("Enter Email Length: "))

prediction=model.predict([[links,caps,length]])

if prediction[0]==1:
    print("Spam Email")
else:
    print("Not Spam Email")
```

Output:



```
1 from sklearn.linear_model import LogisticRegression
2
3 X = [
4     [1,3,120],
5     [6,15,300],
6     [0,1,80],
7     [4,10,250],
8     [2,5,160]
9 ]
10 y = [0,1,0,1,0]
11
12 model = LogisticRegression()
13 model.fit(X,y)
14
15 links = int(input("Enter Number of Links: "))
16 caps = int(input("Enter Number of Capital Words: "))
17 length = int(input("Enter Email Length: "))
18
19 prediction = model.predict([[links,caps,length]])
20
21 if prediction[0]==1:
22     print("Spam Email")
23 else:
24     print("Not Spam Email")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
# saicharan@saicharan-MacBook-Air: ~/code/practice % python3 p3.py
Enter Number of Links: 3
Enter Number of Capital Words: 12
Enter Email Length: 200
Spam Email
```

Result:

Thus, the Logistic Regression model successfully classified the email.

Experiment 4: Linear Regression - Car Resale Price Prediction

Aim:

To predict the resale price of a car using Linear Regression.

Algorithm:

1. Import LinearRegression.
2. Prepare dataset.
3. Train model.
4. Accept user input.
5. Predict resale price.
6. Display the result.

Code:

```
from sklearn.linear_model import LinearRegression

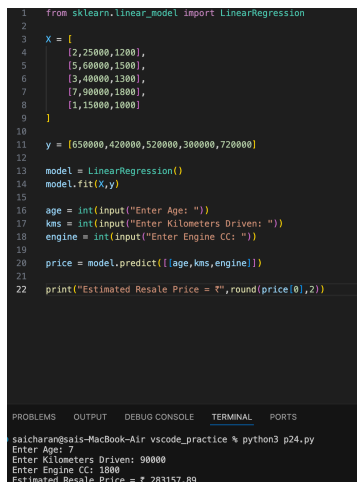
X=[[2,25000,1200],[5,60000,1500],[3,40000,1300],[7,90000,1800],[1,15000,1000]]
y=[650000,420000,520000,300000,720000]

model=LinearRegression()
model.fit(X,y)

age=int(input("Enter Age: "))
kms=int(input("Enter Kilometers Driven: "))
engine=int(input("Enter Engine CC: "))

price=model.predict([[age,kms,engine]])
print("Estimated Resale Price = ₹", round(price[0],2))
```

Output:



```
1 from sklearn.linear_model import LinearRegression
2
3 X = [
4     [2,25000,1200],
5     [5,60000,1500],
6     [3,40000,1300],
7     [7,90000,1800],
8     [1,15000,1000]
9 ]
10
11 y = [650000,420000,520000,300000,720000]
12
13 model = LinearRegression()
14 model.fit(X,y)
15
16 age = int(input("Enter Age: "))
17 kms = int(input("Enter Kilometers Driven: "))
18 engine = int(input("Enter Engine CC: "))
19
20 price = model.predict([[age,kms,engine]])
21
22 print("Estimated Resale Price = ₹",round(price[0],2))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@saicharan-MacBook-Air: ~ % python3 p24.py
Enter Age: 7
Enter Kilometers Driven: 90000
Enter Engine CC: 1800
Estimated Resale Price = ₹ 283157.89
```

Result:

Thus, the resale price was predicted successfully.

Experiment 5: CART Decision Tree - Loan Approval

Aim:

To classify loan applications using the CART Decision Tree algorithm.

Algorithm:

1. Import DecisionTreeClassifier.
2. Prepare dataset.
3. Train the classifier.
4. Accept user input.
5. Predict approval.
6. Display decision rules.

Code:

```
from sklearn.tree import DecisionTreeClassifier, export_text

X=[[45000,720,0],[30000,650,1],[60000,780,0],[28000,620,2],[52000,710,1]]
y=[1,0,1,0,1]

model=DecisionTreeClassifier(criterion="gini")
model.fit(X,y)

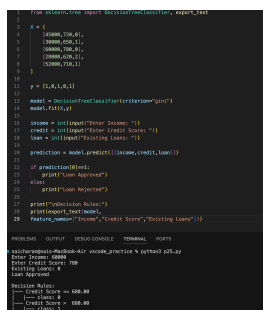
income=int(input("Enter Income: "))
credit=int(input("Enter Credit Score: "))
loan=int(input("Enter Existing Loans: "))

prediction=model.predict([[income,credit,loan]])

if prediction[0]==1:
    print("Loan Approved")
else:
    print("Loan Rejected")

print(export_text(model,feature_names=["Income","Credit Score","Existing Loans"]))
```

Output:



```
1 from sklearn.tree import DecisionTreeClassifier, export_text
2
3 x =
4   [[45000,720,0],
5    [30000,650,1],
6    [60000,780,0],
7    [28000,620,2],
8    [52000,710,1]]
9
10 y = [1,0,1,0,1]
11
12 model = DecisionTreeClassifier(criterion="gini")
13 model.fit(x,y)
14
15 income = int(input("Enter Income: "))
16 credit = int(input("Enter Credit Score: "))
17 loan = int(input("Enter Existing Loans: "))
18
19 prediction = model.predict([[income,credit,loan]])
20
21 if prediction[0]==1:
22     print("Loan Approved")
23 else:
24     print("Loan Rejected")
25
26 print(export_text(model,feature_names=["Income","Credit Score","Existing Loans"]))
```

Python 3.7.4 Shell: Python 3.7.4 Shell [base] x86_64-pc-linux-gnu

Enter Income: 45000
Enter Credit Score: 720
Enter Existing Loans: 0

Loan Approved

Decision Rules:

- Income > 45000.00
- Loan Approved

Result:

Thus, the CART Decision Tree classifier successfully predicted the loan approval status.